

```

import numpy as np
import matplotlib.pyplot as plt

# -----
# Функция и её производная
# -----
def f(x):
    return x**2 - 2*x + np.exp(-x)

def df(x):
    return 2*x - 2 - np.exp(-x)

# -----
# Одна итерация кубической аппроксимации
# Возвращает xm и коэффициенты H(x)
# -----
def cubic_step(a, b):
    x0, x1 = a, b
    delta = x1 - x0

    y0, y1 = f(x0), f(x1)
    dy0, dy1 = df(x0), df(x1)

    A = y0
    B = dy0 + 2*y0 / delta
    C = y1
    D = -dy1 + 2*y1 / delta

    a3 = B - D
    a2 = (A - B*x0 - 2*B*x1) + (C + D*x1 + 2*D*x0)
    a1 = (-2*A + 2*B*x0 + B*x1)*x1 + (-2*C - 2*D*x1 - D*x0)*x0
    a0 = (A - B*x0)*x1**2 + (C + D*x1)*x0**2

    # Минимум H'(x) = 3a3 x^2 + 2a2 x + a1
    if abs(a3) < 1e-14:
        xm = -a1 / (2*a2)
    else:
        disc = a2**2 - 3*a1*a3
        r1 = (-a2 + np.sqrt(disc)) / (3*a3)
        r2 = (-a2 - np.sqrt(disc)) / (3*a3)
        # выбираем корень внутри [x0, x1]
        xm = r1 if x0 < r1 < x1 else r2

    coeffs = (a3, a2, a1, a0)
    return xm, coeffs, (A, B, C, D), (y0, y1, dy0, dy1)

def H(x, coeffs):
    a3, a2, a1, a0 = coeffs
    return a3*x**3 + a2*x**2 + a1*x + a0

```

```

# -----
# Основной алгоритм: генерация последовательности
# -----
def cubic_interpolation(a_init, b_init, eps=1e-4):
    a, b = a_init, b_init
    history = []

    header = "f(xm)"
    print(f"{'Ит':>3} {'a':>12} {'b':>12} {'xm':>12} {'f(xm)':>12} {'header:>12} {'δ':>12}")
    print("-" * 90)

    for i in range(1, 50):
        xm, coeffs, abcd, vals = cubic_step(a, b)
        fpm = df(xm)
        delta = b - a

        print(f"{'i':>3} {'a':>12.8f} {'b':>12.8f} {'xm':>12.8f} "
              f"{'f(xm)':>12.8f} {'fpm':>12.8f} {'delta':>12.8f}")

        history.append({
            'iter': i, 'a': a, 'b': b,
            'xm': xm, 'fxm': f(xm), 'fpm': fpm,
            'delta': delta, 'coeffs': coeffs
        })

    # Критерий останова
    if delta < eps or abs(fpm) < eps:
        print(f"\nОстановка на итерации {i}: "
              f"{'f(xm)':>12.8f} = {'abs(fpm):>12.8f} < ε = {'eps':>12.8f}")
        print(f"{'x*':>12.8f} = {'xm':>12.8f}, {'f(x*)':>12.8f} = {'f(xm)':>12.8f}")
        break

    if fpm < 0:
        a = xm
    else:
        b = xm

    return history

```

```

# -----
# Визуализация
# -----
COLORS = ['#5B4FCF', '#D85A30', '#1D9E75', '#BA7517', '#993556']

def plot_iterations(history, a_init, b_init):
    n = len(history)
    cols = min(n, 3)
    rows = (n + cols - 1) // cols

    fig = plt.figure(figsize=(6*cols, 4.5*rows))
    fig.patch.set_facecolor('#F8F8F6')

    x_global = np.linspace(a_init - 0.05, b_init + 0.05, 400)

    for idx, it in enumerate(history):
        ax = fig.add_subplot(rows, cols, idx + 1)
        ax.set_facecolor('#FAFAF8')
        for spine in ax.spines.values():
            spine.set_linewidth(0.5)
            spine.set_color('#CCCCCC')

        a, b = it['a'], it['b']
        xm = it['xm']
        col = COLORS[idx % len(COLORS)]

        # Исходная функция
        ax.plot(x_global, f(x_global),
                color='#2C2C2A', linewidth=1.8,
                label=f' $f(x) = x^2 - 2x + e^{-x}$ ', zorder=3)

        # Аппроксимирующий кубический многочлен H(x) на [a, b]
        x_loc = np.linspace(a, b, 300)
        ax.plot(x_loc, H(x_loc, it['coeffs']),
                color=col, linewidth=2.0, linestyle='--',
                label=f'H(x) — кубич. аппрокс.', zorder=4)

        # Текущий отрезок [a, b]
        ax.axvspan(a, b, alpha=0.08, color=col, zorder=1)
        ax.axvline(a, color=col, linewidth=0.8, linestyle=':', alpha=0.7)
        ax.axvline(b, color=col, linewidth=0.8, linestyle=':', alpha=0.7)

        # Точка минимума xm
        ax.scatter([xm], [f(xm)], color=col, s=60, zorder=6,
                  edgecolors='white', linewidth=1.2)
        ax.axvline(xm, color=col, linewidth=1.0, alpha=0.5, linestyle='-.')

```

```

# Производные на концах (касательные-стрелки)
for xp in [a, b]:
    slope = df(xp)
    dx = (b - a) * 0.08
    ax.annotate(" xy=(xp + dx, f(xp) + slope*dx),
                xytext=(xp, f(xp)),
                arrowprops=dict(arrowstyle='->', color='#888780',
                                lw=1.2), zorder=5)

ax.set_title(f'Итерация {it['iter']} "
             f"[{a:.5f}, {b:.5f}]\n"
             f"x_m = {xm:.6f}, f'(x_m) = {it['fpm']:.2e}",
             fontsize=10, color='#2C2C2A', pad=6)
ax.set_xlabel('x', fontsize=9, color='#5F5E5A')
ax.set_ylabel('y', fontsize=9, color='#5F5E5A')
ax.tick_params(labelsize=8, colors='#5F5E5A')
ax.legend(fontsize=8, framealpha=0.7, loc='upper right')
ax.grid(True, linewidth=0.4, color='#D3D1C7', alpha=0.7)

fig.suptitle('Метод кубической интерполяции\n'
             'f(x) = x^2 - 2x + e^-x, [a,b] = [1, 1.5], ε = 0.0001',
             fontsize=13, color='#2C2C2A', y=1.01, fontweight='normal')
plt.tight_layout()
plt.savefig('cubic_interpolation.png',
            dpi=160, bbox_inches='tight', facecolor=fig.get_facecolor())
print("\nГрафик сохранён: cubic_interpolation.png")
plt.show()

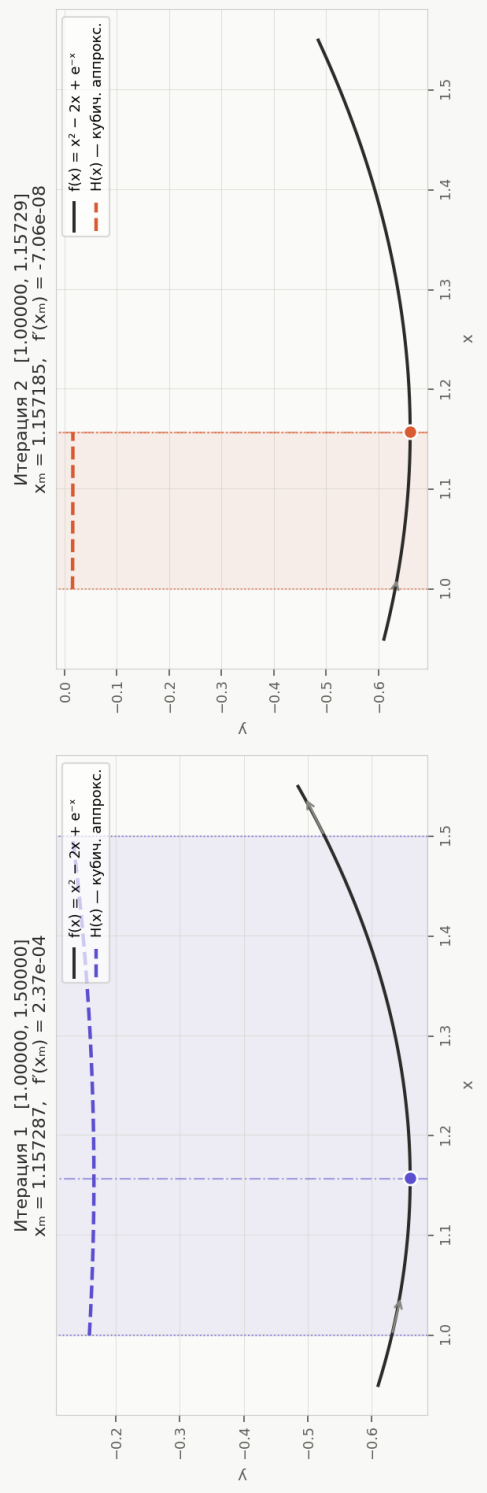
# -----
# Запуск
# -----
if __name__ == '__main__':
    EPS = 1e-4
    A0, B0 = 1.0, 1.5

    print("=" * 90)
    print(" Метод кубической аппроксимации")
    print(f" f(x) = x^2 - 2x + e^(-x), [a,b] = [{A0}, {B0}], ε = {EPS}")
    print("=" * 90 + "\n")

    history = cubic_interpolation(A0, B0, eps=EPS)
    plot_iterations(history, A0, B0)

```

Метод кубической интерполяции
 $f(x) = x^2 - 2x + e^{-x}$, $[a,b] = [1, 1.5]$, $\varepsilon = 0.0001$



```
import numpy as np
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Callable, List, Optional
from scipy.optimize import minimize_scalar
```

```
EPS = 1e-4
```

```
# =====
# Д32.3 — ОБЪЕКТНАЯ МОДЕЛЬ
# =====
```

```
@dataclass
class OptimizationResult:
    """Результат работы метода (хранит историю приближений)."""
    minimum: np.ndarray
    fun_value: float
    history: List[np.ndarray]
    iterations: int

class ObjectiveFunction:
    """
    Целевая функция с градиентом и (опционально) матрицей Гессе.
    Агрегирует callable-объекты func, grad, hessian.
    """
    def __init__(
        self,
        func: Callable,
        grad: Callable,
        hessian: Optional[np.ndarray] = None
    ):
        self._func = func
        self._grad = grad
        self._hessian = hessian

    def __call__(self, x: np.ndarray) -> float:
        return self._func(x)

    def gradient(self, x: np.ndarray) -> np.ndarray:
        return self._grad(x)

    @property
    def hessian(self) -> Optional[np.ndarray]:
        return self._hessian
```

```

class Optimizer(ABC):
    """
    Абстрактный класс оптимизатора.
    Использует ObjectiveFunction (зависимость/использование).
    Конкретные классы реализуют метод step().
    """
    def __init__(self, obj: ObjectiveFunction, eps: float = EPS):
        self.obj = obj      # агрегация
        self.eps = eps

    @abstractmethod
    def step(self, x: np.ndarray) -> np.ndarray:
        """Один шаг метода — возвращает следующее приближение."""

    def optimize(self, x0: np.ndarray) -> OptimizationResult:
        """Итеративная оптимизация до выполнения критерия останова."""
        x = np.array(x0, dtype=float)
        history = [x.copy()]
        while True:
            x_new = self.step(x)
            history.append(x_new.copy())
            if np.linalg.norm(self.obj.gradient(x_new)) < self.eps:
                break
            x = x_new
        return OptimizationResult(
            minimum=x_new,
            fun_value=self.obj(x_new),
            history=history,
            iterations=len(history) - 1,
        )

class CoordinateDescent(Optimizer):
    """
    Метод покоординатного спуска.

    На каждой итерации последовательно минимизирует функцию
    по каждой координате при фиксированных остальных.
    Наследует Optimizer, реализует step().
    """
    def step(self, x: np.ndarray) -> np.ndarray:
        x_new = x.copy()
        for i in range(len(x_new)):
            def phi(t, idx=i):
                xc = x_new.copy()
                xc[idx] = t
                return self.obj(xc)
            res = minimize_scalar(phi, method='brent')
            x_new[i] = res.x
        return x_new

```

```

class GradientDescent(Optimizer):
    """
    Метод градиентного спуска с постоянным шагом.
     $x^{k+1} = x^k - lr * \nabla f(x^k)$ 
    Наследует Optimizer, реализует step().
    """
    def __init__(self, obj: ObjectiveFunction, eps: float = EPS, lr: float = 0.05):
        super().__init__(obj, eps)
        self.lr = lr # шаг (learning rate)

    def step(self, x: np.ndarray) -> np.ndarray:
        return x - self.lr * self.obj.gradient(x)

```

```

class SteepestDescent(Optimizer):
    """
    Метод наискорейшего спуска (оптимальный шаг).
    Оптимальный шаг для квадратичной функции:
     $\alpha^* = (g^T g) / (g^T H g)$ 
    Для произвольной функции — точная одномерная минимизация.
    Наследует Optimizer, реализует step().
    """
    def step(self, x: np.ndarray) -> np.ndarray:
        g = self.obj.gradient(x)
        if self.obj.hessian is not None:
            # Аналитический оптимальный шаг (квадратичный случай)
            Hg = self.obj.hessian @ g
            alpha = float(g @ g) / float(g @ Hg)
        else:
            # Численная одномерная минимизация
            res = minimize_scalar(lambda a: self.obj(x - a * g),
                                bounds=(0, 10), method='bounded')
            alpha = res.x
        return x - alpha * g

```

```

# ФУНКЦИЯ ЛР4 (вариант 5)

```

```

def f_lr4(x):
    x1, x2 = x[0], x[1]
    return 3*x1**2 + 2*x1*x2 + 3*x2**2 + 6*x1 + 18*x2 + 27

```

```
def grad_f_lr4(x):
    x1, x2 = x[0], x[1]
    return np.array([6*x1 + 2*x2 + 6,
                     2*x1 + 6*x2 + 18])
```

```
H_LR4 = np.array([[6., 2.],
                  [2., 6.]])
```

```
# =====
# ДЗ2.1 — ВИЗУАЛИЗАЦИЯ
# =====
```

```
def plot_contours_and_path(
    result: OptimizationResult,
    func_vec: Callable,
    title: str,
    xlim=(-1.5, 3.5),
    ylim=(-5.0, 4.5),
):
    history = np.array(result.history)

    x1_grid = np.linspace(*xlim, 500)
    x2_grid = np.linspace(*ylim, 500)
    X1, X2 = np.meshgrid(x1_grid, x2_grid)
    Z = func_vec(X1, X2)

    levels = sorted({float(f_lr4(pt)) for pt in result.history})

    fig, ax = plt.subplots(figsize=(7, 6))
    cs = ax.contour(X1, X2, Z, levels=levels, cmap='RdYlBu_r', linewidths=0.9)
    ax.clabel(cs, fmt='%.2f', fontsize=7, inline=True)

    # Ломаная
    ax.plot(history[:, 0], history[:, 1],
            'o-', color='crimson', linewidth=1.6, markersize=4,
            label=f'Приближения ({result.iterations} итер.)')
    ax.plot(*history[0], 'gs', markersize=9, zorder=5, label='Старт')
    ax.plot(*history[-1], 'r*', markersize=14, zorder=5, label='Минимум')

    ax.set_xlabel('$x_1$', fontsize=11)
    ax.set_ylabel('$x_2$', fontsize=11)
    ax.set_title(title, fontsize=12)
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.25)
    plt.tight_layout()
    return fig
```

```
# =====  
# ДЗ2.2 — МЕТОД НЬЮТОНА  
# =====
```

```
def z_dz22(xy):  
    x, y = float(xy[0]), float(xy[1])  
    return x**3 - 3*x**2 + y**3 + 4*y**2 + 4*y
```

```
def grad_z_dz22(xy):  
    x, y = float(xy[0]), float(xy[1])  
    return np.array([3*x**2 - 6*x,  
                    3*y**2 + 8*y + 4])
```

```
def hess_z_dz22(xy):  
    x, y = float(xy[0]), float(xy[1])  
    return np.array([[6*x - 6, 0. ],  
                    [0., 6*y + 8]])
```

```
def newton_method(func, grad, hess, x0, eps=EPS):  
    x = np.array(x0, dtype=float)  
    history = [x.copy()]
```

```
    print(f"\n{'k':>4} {'x1':>14} {'x2':>14} "  
          f"{'∂z/∂x':>13} {'∂z/∂y':>13}")  
    print("—" * 62)
```

```
    k = 0  
    while True:  
        g = grad(x)  
        print(f"{'k':>4} {'x[0]:>14.8f} {'x[1]:>14.8f} "  
              f"{'g[0]:>13.8f} {'g[1]:>13.8f}")  
        if max(abs(g[0]), abs(g[1])) < eps:  
            print("—" * 62)  
            print(f"Критерий останова выполнен на итерации {k}.")  
            break  
        H = hess(x)  
        x = x - np.linalg.inv(H) @ g  
        k += 1  
        history.append(x.copy())
```

```
    print(f"\nРезультат: x* = ({x[0]:.8f}, {x[1]:.8f})")  
    print(f"          z(x*) = {func(x):.8f}")  
    return x, history
```

```

# =====
# MAIN
# =====

if __name__ == "__main__":

    print("=" * 62)
    print("ЛР4 Вариант 5")
    print("f(x1,x2) = 3x12 + 2x1x2 + 3x22 + 6x1 + 18x2 + 27")
    print("Начальная точка: (2, 3), ε = 0.0001")
    print("=" * 62)

    obj_lr4 = ObjectiveFunction(f_lr4, grad_f_lr4, H_LR4)
    x0_lr4 = np.array([2.0, 3.0])

    def f_vec_lr4(X1, X2):
        return 3*X1**2 + 2*X1*X2 + 3*X2**2 + 6*X1 + 18*X2 + 27

    methods = {
        "Покоординатный спуск": CoordinateDescent(obj_lr4),
        "Градиентный спуск": GradientDescent(obj_lr4, lr=0.07),
        "Наискорейший спуск": SteepestDescent(obj_lr4),
    }

    for name, method in methods.items():
        res = method.optimize(x0_lr4)
        print(f"\n{name}:")
        print(f"Итераций : {res.iterations}")
        print(f"Минимум : ({res.minimum[0]:.7f}, {res.minimum[1]:.7f})")
        print(f"f(x*) : {res.fun_value:.7f}")
        plot_contours_and_path(res, f_vec_lr4, name)

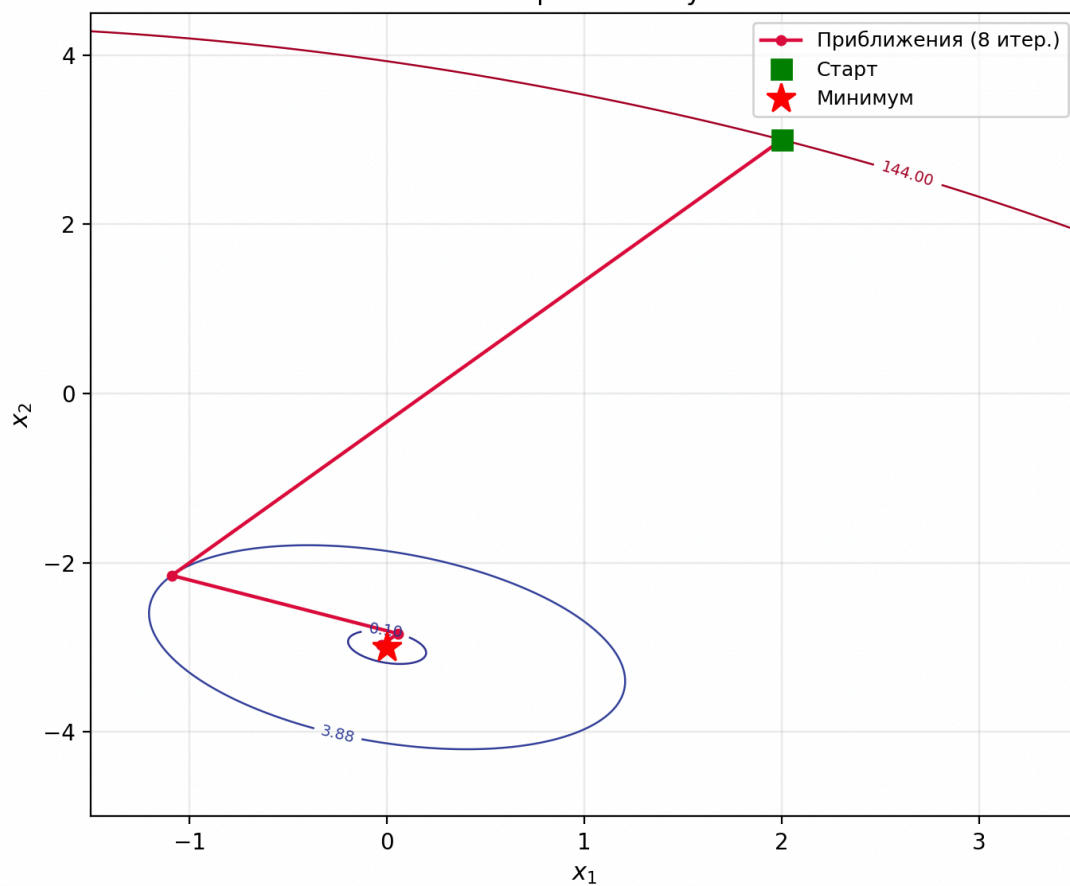
# — Д32.2 —=====
print("\n" + "=" * 62)
print("Д32.2 Метод Ньютона (вариант 5)")
print("z(x,y) = x3 - 3x2 + y3 + 4y2 + 4y")
print("Начальная точка: (1.5, -0.5), ε = 0.0001")
print("=" * 62)

newton_method(z_dz22, grad_z_dz22, hess_z_dz22,
              x0=[1.5, -0.5], eps=EPS)

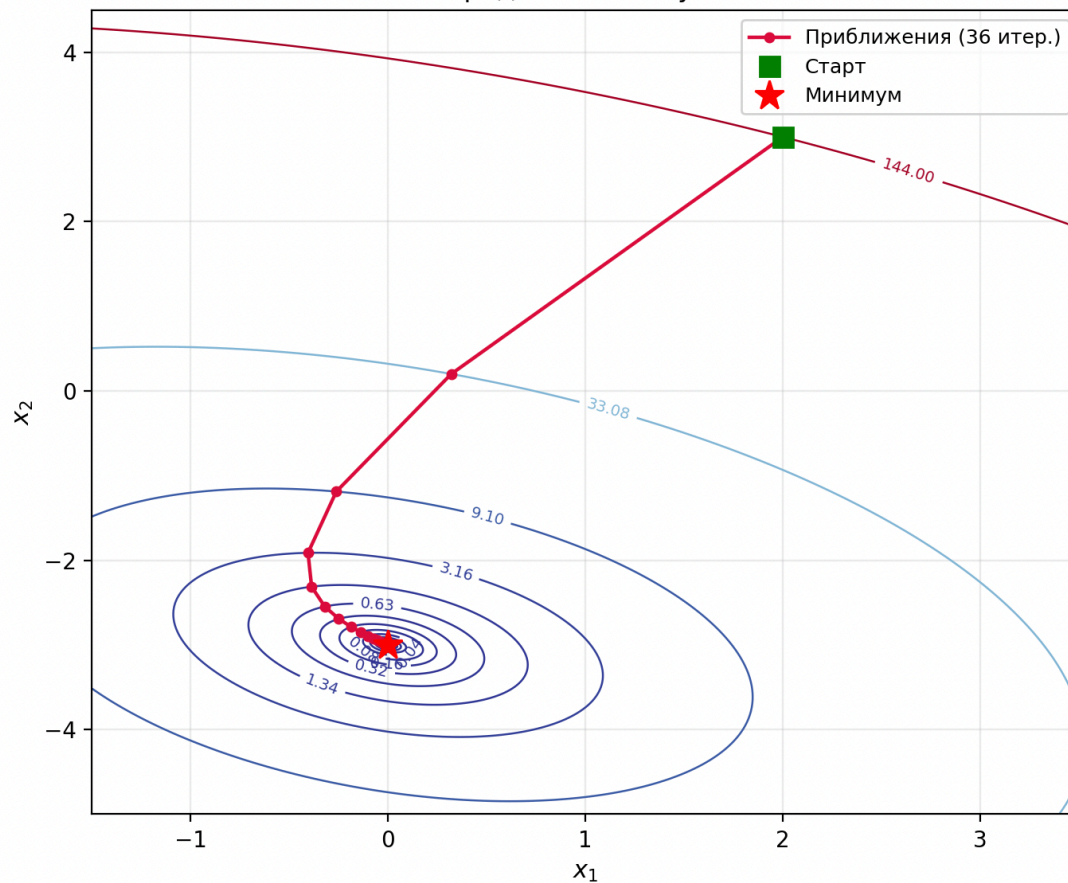
plt.show()

```

Наискорейший спуск



Градиентный спуск



Покоординатный спуск

